Qualité de développement

Système de gestion de version : Git

Introduction

Le but des systèmes de gestion de version (*version control systems*) est principalement double :

1. Garder un historique des différentes versions d'un projet (au sens large)
2. Faciliter la collaboration entre plusieurs personne travaillant sur le même projet

Leur rôle est d'automatiser ces tâches et de les rendre efficaces.

Historique des versions

L'historique des versions est un enregistrement des modifications apportées à un projet au fil du temps. Il permet de revenir à une version antérieure du projet, de comparer deux versions, de voir qui a fait quoi, etc.

Collaboration

La collaboration est facilitée par la possibilité de travailler sur le même projet en même temps, sans risque de conflit. Les modifications sont enregistrées dans un dépôt centralisé, et chaque personne peut récupérer les modifications des autres.

Git

Git est un système de gestion de version distribué. Il permet de gérer des projets de toutes tailles, de collaborer avec des milliers de personnes, et de gérer des projets de manière efficace.

Nous allons maintenant présenter quelques notions de base de git.

Dépôt

Un dépôt est un ensemble de fichiers et de répertoires, ainsi que l'historique des modifications apportées à ces fichiers. Il est possible de créer un dépôt à partir d'un dossier existant, ou de cloner un dépôt existant.

Localement, un dépôt est un dossier contenant un dossier `.git` qui contient l'historique des modifications apportées au dépôt.

Remarque : l'accès et la modification du dépôt doit se faire (sauf rares exceptions) via les commandes git.

Commit

Un **commit** est une modification apportée à un dépôt. Il est possible de faire plusieurs commits dans un dépôt, et de revenir à une version antérieure du dépôt en annulant les commits.

Un commit est composé de :

- Un message de commit
- Un auteur
- La date de création
- Un identifiant unique
- Les modifications apportées au dépôt

Index

L'index est un espace temporaire contenant les modifications prêtes à être *commitées*. Ces modifications peuvent être :

- création de fichier
- modification de fichier
- suppression de fichier

Verbes

Les verbes de git sont les commandes permettant de manipuler le dépôt. Les verbes les plus utilisés sont :

- `git init` : créer un dépôt
- `git clone` : cloner un dépôt
- `git add` : ajouter des modifications à l'index
- `git commit` : créer un commit
- `git push` : envoyer les commits sur le dépôt distant
- `git pull` : récupérer les commits du dépôt distant
- `git status` : afficher l'état du dépôt
- `git log` : afficher l'historique des commits

Identifiants (à faire une seule fois)

Pour pouvoir utiliser git, il faut configurer son nom et son email. Cela permet de savoir qui a fait quoi.

```
git config --global user.name "John Doe"
git config --global user.email "john.doe@email.com"
```

Aide

Pour obtenir de l'aide sur une commande, il suffit d'ajouter `--help` à la fin de la commande.

```
git add --help
```

Création d'un dépôt git

Pour mettre en place un dépôt, il y a principalement deux manières :

1. Faire d'un répertoire existant un dépôt Git (ce qui est expliqué en cours)

2. **Cloner** un dépôt existant (*quelque part*) (ce qui sera fait en TD)

Création d'un dépôt git local

Pour créer un dépôt git, il faut se placer dans le répertoire à versionner et exécuter la commande `git init`.

```
% cd /home/user/projet
% git init
Initialized empty Git repository in
/Users/njozefow/git/enseignement/but1/coursgit/.git/
% ls -la
total 0
drwxr-xr-x  3 owner  staff  102  9 nov 10:45 .
drwxr-xr-x  5 owner  staff  170  9 nov 10:45 ..
drwxr-xr-x  9 owner  staff  306  9 nov 10:45 .git
```

La commande crée un répertoire `.git`. Le répertoire `.git` contient l'historique des modifications apportées au dépôt.

Remarque : si le répertoire n'est pas vide, il faut d'abord ajouter les fichiers au dépôt avec `git add` (cf ci-dessous).

Modification d'un dépôt git local

Une modification peut être :

- l'ajout d'un nouveau fichier
- la suppression d'un fichier
- la modification du contenu d'un fichier

Ces états sont établis par rapport au dernier commit.

Tester l'état des fichiers

Pour voir l'état des fichiers, il faut exécuter la commande `git status`.

```
% git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Ajout d'un nouveau fichier sous suivi de version

On crée un nouveau fichier dans le projet :

```
echo 'console.log("Hello, world!");' > main.ts
```

On peut voir que le fichier n'est pas suivi par git :

```
% git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        main.ts

nothing added to commit but untracked files present (use "git add" to track)
```

Le fichier est marqué comme non suivi (*untracked*). Cela signifie qu'il ne sera pas pris en compte lors des commits et que ses modifications ne sont pas suivi.

Pour ajouter le fichier à l'index, il faut exécuter la commande `git add`.

```
% git add main.ts
% git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   main.ts
```

On peut ajouter d'autres fichiers. Par exemple, ajoutons l'ensemble des fichiers nécessaires à la mise en place d'un environnement de développement TypeScript. On a donc maintenant un répertoire avec les fichiers suivants :

```
% ls -la
total 48
drwxr-xr-x   8 owner  staff    256 31 jan 11:47 .
drwxr-xr-x@  9 owner  staff    288 31 jan 11:33 ..
-rw-r--r--   1 owner  staff    867 25 jan 08:33 .eslintrc.json
drwxr-xr-x  10 owner  staff    320 31 jan 11:42 .git
drwxr-xr-x   4 owner  staff    128 25 jan 08:45 .vscode
-rw-r--r--   1 owner  staff     30 31 jan 11:39 main.ts
-rw-r--r--   1 owner  staff    304 25 jan 08:32 package.json
-rw-r--r--   1 owner  staff  10586 30 jan 15:02 tsconfig.json
```

L'état du dépôt est le suivant :

```
% git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   main.ts

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .eslintrc.json
        .vscode/
        package.json
        tsconfig.json
```

On peut remarquer que les répertoires sont pris en compte comme des fichiers. Cela revient à suivre l'ensemble des fichiers contenus dans le répertoire.

Si on veut suivre les fichiers ajoutés, il faut les ajouter soit un par un, soit en utilisant le caractère `*` pour indiquer un ensemble de fichiers.

```
% git add .eslintrc.json
% git add .vscode/*
% git add package.json
% git add tsconfig.json
% git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .eslintrc.json
        new file:   .vscode/extensions.json
        new file:   .vscode/settings.json
        new file:   main.ts
        new file:   package.json
        new file:   tsconfig.json
```

Commit

Pour enregistrer les modifications, il faut exécuter la commande `git commit`.

```
% git commit -m "Initial commit"
[main (root-commit) 2a69466] Initial commit
 6 files changed, 165 insertions(+)
 create mode 100644 .eslintrc.json
 create mode 100644 .vscode/extensions.json
 create mode 100644 .vscode/settings.json
 create mode 100644 main.ts
 create mode 100644 package.json
 create mode 100644 tsconfig.json
```

L'option `-m` permet de définir un message de commit. Le message de commit est obligatoire. Il est conseillé de définir un message de commit explicite. Le message de commit est utilisé pour décrire les modifications apportées au projet. Vous pouvez trouver des conseils pour rédiger un bon message de commit ici.

On peut voir que le commit a été effectué avec succès. Si on regarde l'état du dépôt, on peut voir que le dépôt est propre et à jour.

```
% git status
On branch main
nothing to commit, working tree clean
```

Historique des commits

Pour voir l'historique des commits, il faut exécuter la commande `git log`.

```
% git log
commit 2a694664c6dd571f1f5988976d14075c8e2261ea (HEAD -> main)
Author: John Doe <john.doe@email.com>
Date:   Tue Jan 31 11:53:04 2023 +0100

    Initial commit
```

Les informations affichées sont les suivantes :

- `commit` : identifiant du commit
- `Author` : auteur du commit
- `Date` : date du commit
- `message` : message du commit

Les nouvelles modifications sont prises en compte après le commit. Par exemple, si on ajoute le fichier `README.md`, le fichier n'est pas pris en compte dans le commit précédent.

```
% echo "Cours Git" > README.md
% git status
On branch main
```

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present (use "git add" to track)
```

Pour que `README.md` soit pris en compte dans les futurs commits, il faut l'ajouter à l'index.

```
% git add README.md
% git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   README.md
```

On peut voir que le fichier `README.md` est ajouté à l'index. Si on fait un nouveau commit, le fichier `README.md` sera pris en compte.

```
% git commit -m "feat: Add README.md"
[main cd5a979] feat: Add README.md
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
% git status
On branch main
nothing to commit, working tree clean
% git log
commit cd5a9792b531a17c6bcdd6b12bf2bf24c5c1a702 (HEAD -> main)
Author: John Doe <john.doe@email.com>
Date:   Tue Jan 31 12:03:11 2023 +0100

    feat: Add README.md

commit 2a694664c6dd571f1f5988976d14075c8e2261ea
Author: John Doe <john.doe@email.com>
Date:   Tue Jan 31 11:53:04 2023 +0100

    Initial commit
```

Remarque : dans le cours, on peut avoir l'impression que dès que l'on fait une modification sur un fichier, on fait un commit. Cela n'est pas la pratique habituelle. En effet, on fait des commits lorsque l'on a des modifications significatives. Par exemple, on peut faire un commit lorsque l'on a terminé une fonctionnalité ou une tâche.

Modification d'un fichier

On peut modifier un fichier. Par exemple, on peut modifier le fichier `main.ts` en ajoutant une ligne.

```
% echo "console.log("Hello Git");" >> main.ts;
```

Si on regarde l'état du dépôt, on peut voir que le fichier `main.ts` a été modifié.

```
% git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   main.ts

no changes added to commit (use "git add" and/or "git commit -a")
```

On peut voir que le fichier `main.ts` a été modifié mais n'est pas ajouté à l'index. Pour ajouter le fichier à l'index, il faut exécuter la commande `git add`.

```
% git add main.ts
% git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   main.ts
```

On peut donc maintenant faire un commit pour enregistrer la modification du fichier `main.ts`.

```
% git commit -m "feat: Modify main.ts"
[main a3e34ae] feat: Modify main.ts
 1 file changed, 1 insertion(+)
% git status
On branch main
nothing to commit, working tree clean
% git log
commit a3e34ae7b78c88f572a671e93c32d63d5bbef060 (HEAD -> main)
Author: John Doe <john.doe@email.com>
Date:   Tue Jan 31 12:10:03 2023 +0100

    feat: Modify main.ts

commit cd5a9792b531a17c6bcdd6b12bf2bf24c5c1a702
Author: John Doe <john.doe@email.com>
Date:   Tue Jan 31 12:03:11 2023 +0100

    feat: Add README.md

commit 2a694664c6dd571f1f5988976d14075c8e2261ea
```

```
Author: John Doe <john.doe@email.com>
Date:   Tue Jan 31 11:53:04 2023 +0100
```

Suppression d'un fichier

On peut supprimer un fichier. Par exemple, on peut supprimer le fichier `main.ts`.

```
% rm main.ts
% git status
On branch main
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    main.ts

no changes added to commit (use "git add" and/or "git commit -a")
```

On peut voir que le fichier `main.ts` a été supprimé mais n'est pas ajouté à l'index. Pour supprimer le fichier à l'index, il faut exécuter la commande `git rm`.

```
% git rm main.ts
rm 'main.ts'
% git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    main.ts
```

Remarque : on peut utiliser directement la commande `git rm` pour supprimer un fichier et l'ajouter à l'index.

Renommer un fichier

On peut renommer un fichier. Par exemple, on peut renommer le fichier `README.md` en `README.txt`.

```
% mv README.md README.txt
% git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    main.ts

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    README.md
```

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.txt
```

On peut voir que le fichier `README.md` a été supprimé et le fichier `README.txt` a été ajouté. Pour les mettre à l'index, il faut exécuter la commande `git add` pour l'un et `git rm` pour l'autre. On peut faire les deux en même temps avec la commande `git mv`.

Commençons par annuler la modification non indexée sur `README.md` :

```
% git restore README.md
```

pour `README.txt`, il faut juste le supprimer puisqu'il n'est pas suivi par Git.

```
% rm README.txt
```

Nous pouvons maintenant renommer le fichier `README.md` en `README.txt` en utilisant la commande `git mv` et le mettre directement à l'index.

```
% git mv README.md README.txt
% git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        renamed:    README.md -> README.txt
        deleted:    main.ts
```

Cependant, nous allons voir qu'il y a une manière de prendre en compte les modifications non indexées lorsque l'on fait un commit.

Ne pas suivre des fichiers ou répertoires

Supposons que l'on installe l'environnement de développement :

```
% npm install

added 151 packages, and audited 152 packages in 10s

34 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Cela a pour effet de créer le répertoire `node_modules` qui contient les dépendances du projet ainsi que le fichier `package-lock.json` qui contient les versions des dépendances. Ils ne sont pas suivis par Git car on ne les a pas ajoutés à l'index.

```
% git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        renamed:    README.md -> README.txt
        deleted:    main.ts

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        node_modules/
        package-lock.json
```

On ne veut pas suivre ce répertoire avec Git car il est trop volumineux et peut-être facilement recréer à partir de `package.json`. On pourrait le laisser ainsi mais il vaut mieux faire en sorte que Git ne les prenne pas en compte du tout. Pour cela, il faut créer un fichier `.gitignore` à la racine du projet et y ajouter les noms du répertoire `node_modules` et du fichier `package-lock.json`.

```
% echo "node_modules" >> .gitignore
% echo "package-lock.json" >> .gitignore
```

On peut voir que Git ne les prend plus en compte.

```
% git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        renamed:    README.md -> README.txt
        deleted:    main.ts

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
```

Commit avec modifications non indexées

On peut voir que le fichier `.gitignore` n'est pas ajouté à l'index. On pourrait le faire avec la commande `git add`. Mais si pour un commit, on a plusieurs modifications non indexées, il est plus pratique de les prendre en compte toutes en même temps. Supposons ici que l'on modifie le fichier `README.txt` et que l'on crée un répertoire `src` avec un fichier `main.ts` :

```
% echo "Hello World" >> README.txt
% mkdir src
% echo "console.log('Hello World')" >> src/main.ts
```

On a trois modifications non indexées que l'on souhaite prendre en compte dans le commit :

```
% git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        renamed:    README.md -> README.txt
        deleted:    main.ts

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        src/
```

Remarque : Git prend en compte la création du répertoire `src` et à travers lui, la création du fichier `main.ts`.

Plutôt que de faire `git add` pour chaque fichier, on peut utiliser la commande `git commit -a` qui va prendre en compte toutes les modifications non indexées.

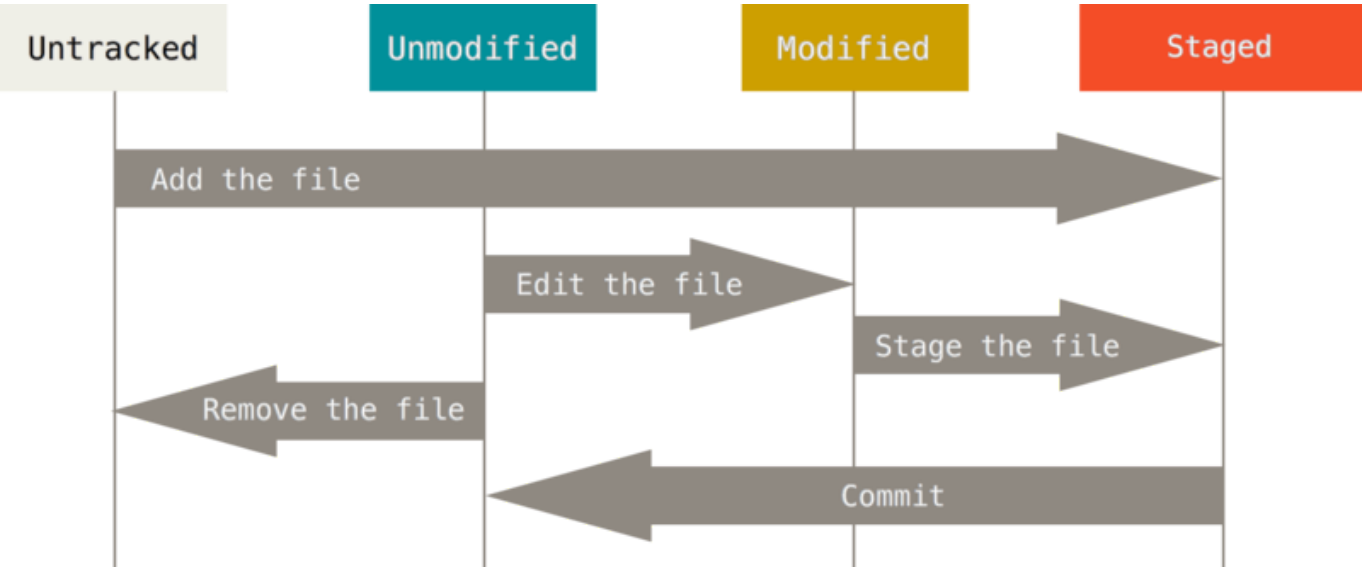
```
% git commit -a -m "feat: add src directory + rename README.md + modify README.txt +
deleted main.ts"
[main 4d48afd] feat: add src directory + rename README.md + modify README.txt + deleted
main.ts
 3 files changed, 2 insertions(+), 3 deletions(-)
 delete mode 100644 README.md
 create mode 100644 README.txt
 delete mode 100644 main.ts
% git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        src/

nothing added to commit but untracked files present (use "git add" to track)
```

On voit que la modification du fichier `README.txt` est prise en compte dans le commit. Mais le fichier `.gitignore` n'est pas ajouté à l'index. Il en est de même pour le répertoire `src`. L'option `-a` met à l'indexe uniquement pour les fichiers qui sont déjà suivis. Pour les nouveaux fichiers, il faut donc d'abord les ajouter à l'index avec `git add` :

```
% git add .gitignore src
% git commit -a -m "feat: add src directory"
 2 files changed, 3 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 src/main.ts
% git status
On branch main
nothing to commit, working tree clean
```

Cycle des états d'un fichier



Revenir en arrière

Supposons que l'on efface le fichier `main.ts` et que l'on fasse un commit :

```
% git rm src/main.ts
rm 'src/main.ts'
% git commit -m "feat: delete main.ts"
[main 1b0b0a6] feat: delete main.ts
 1 file changed, 1 deletion(-)
 delete mode 100644 src/main.ts
```

On se rend compte par la suite que l'on a fait une erreur et qu'il faut récupérer le fichier `main.ts`. On peut le faire notamment en revenant au commit précédent :

```
% git reset --hard HEAD^
HEAD is now at 4d48afd feat: add src directory + rename README.md + modify README.txt +
deleted main.ts
% ls
README.txt              node_modules            package-lock.json       package.json
src                     tsconfig.json
% git status
On branch main
nothing to commit, working tree clean
```

On peut voir via `git log` que le dernier commit a bien été supprimé. Il existe d'autres manières de revenir en arrière au-delà du dernier commit qui seront vu ultérieurement.

Utilisation de Git avec un serveur distant

Jusque maintenant, on a utilisé Git en local. On peut aussi utiliser Git avec un serveur distant. Cela permet de sauvegarder les modifications sur un serveur distant et de pouvoir travailler à plusieurs sur un même projet. Il existe de nombreux serveurs Git. On peut citer par exemple GitHub, GitLab, Bitbucket, etc. Dans le cours, nous allons utiliser le serveur gitlab de l'Université de Lorraine (https://gitlab.univ-lorraine.fr). Vous pouvez vous y connecter avec vos identifiants universitaires.

Remarque : vous ne pouvez disposer que de 5 dépôts sur le serveur GitLab de l'Université de Lorraine. Si vous en avez déjà 5, vous ne pourrez pas en créer de nouveaux et il vous faudra en supprimer un pour en créer un nouveau.

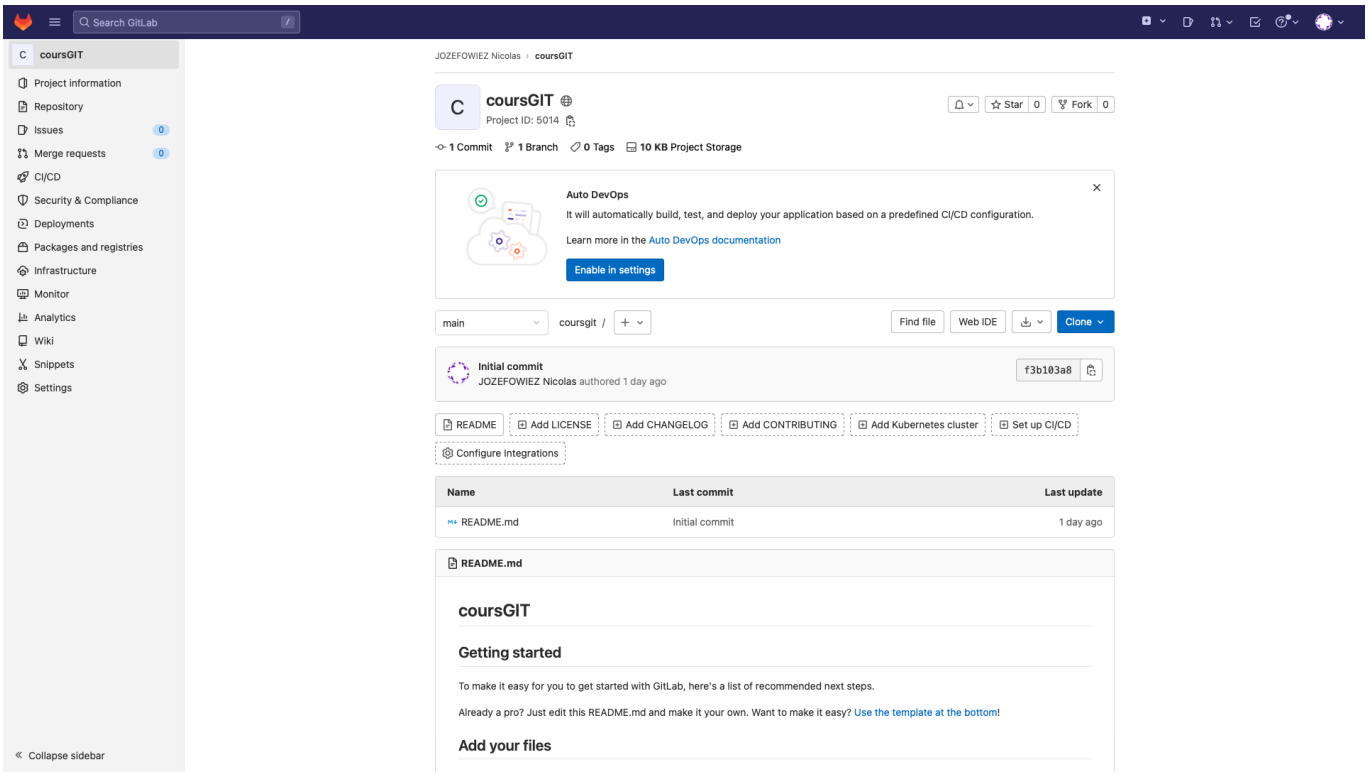
Création d'un dépôt distant

Une fois connecter sur le serveur GitLab, on peut créer un nouveau dépôt en cliquant sur `New project` puis choisir `Create blank project`. On peut alors lui donner un nom. On peut choisir la visibilité :

- `private` : il faut être invité pour voir le projet
- `public` : tout le monde peut voir et accéder au projet

Ici, nous allons le rendre `public` et l'appeler `coursGIT`. On va aussi décocher la case `Initialize repository with a README` car nous avons déjà un fichier `README.txt` dans notre dépôt local.

On obtient alors une page avec les informations du dépôt. On peut voir l'URL du dépôt qui est `https://gitlab.univ-lorraine.fr/username/coursGIT.git`.



Lier le dépôt local au dépôt distant

Pour lier le dépôt local au dépôt distant, on utilise la commande `git remote add` :

```
% git remote add origin https://gitlab.univ-lorraine.fr/username/coursGIT.git
```

On peut vérifier que le dépôt distant a bien été ajouté avec la commande `git remote -v` :

```
% git remote -v
origin  https://gitlab.univ-lorraine.fr/username/coursGIT.git (fetch)
origin  https://gitlab.univ-lorraine.fr/username/coursGIT.git (push)
```

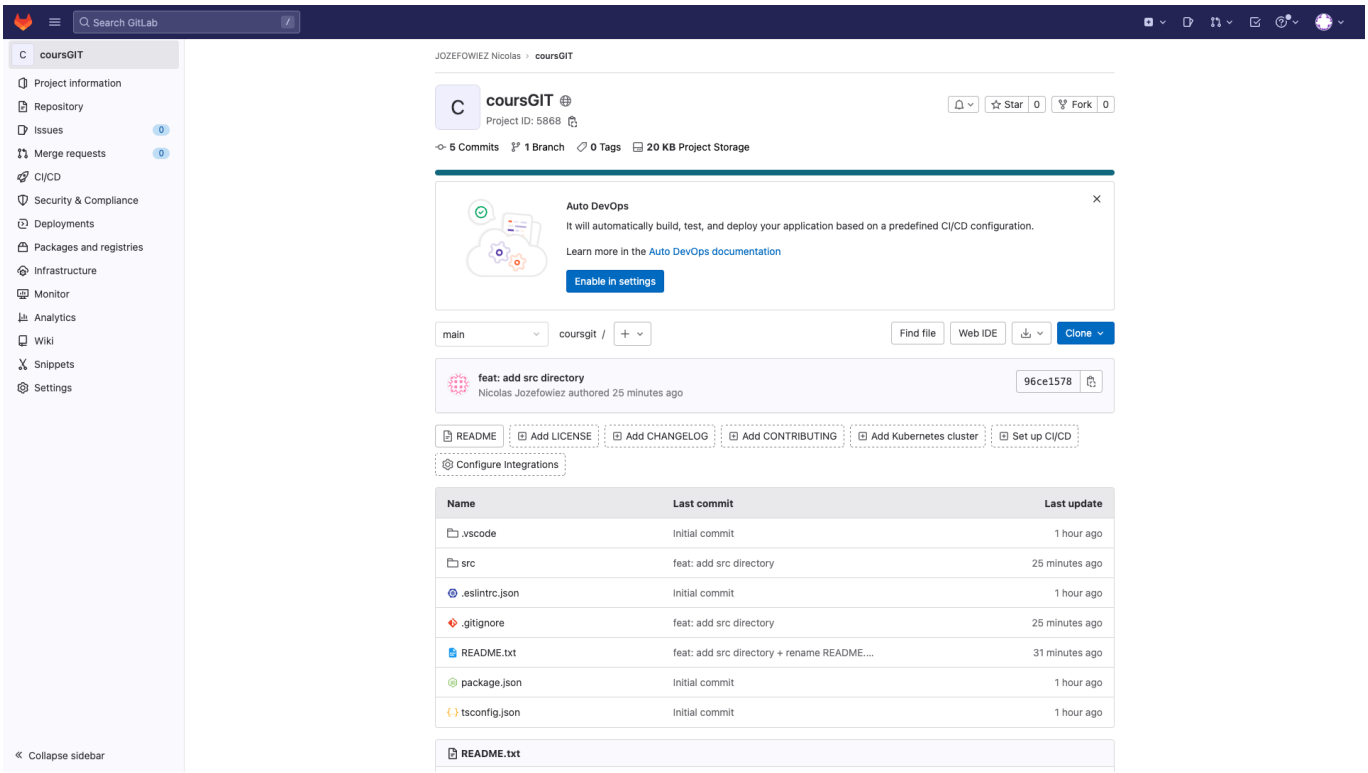
Pousser les modifications sur le dépôt distant

Les modifications locales ne sont pas encore sur le dépôt distant. Pour les pousser, on utilise la commande `git push` :

```
% git push -u origin main
Enumerating objects: 23, done.
Counting objects: 100% (23/23), done.
Delta compression using up to 10 threads
Compressing objects: 100% (17/17), done.
Writing objects: 100% (23/23), 5.64 KiB | 5.64 MiB/s, done.
Total 23 (delta 4), reused 0 (delta 0), pack-reused 0
To https://gitlab.univ-lorraine.fr/jozefowi1/coursgit.git
```

```
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

On peut voir que le dépôt distant a bien été mis à jour et que l'ensemble des commits que l'on a fait sur le dépôt local ont été poussés sur le dépôt distant (cliquer sur `x commits`):



On peut aussi voir que le dépôt local a été mis à jour avec la commande `git status` :

```
% git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

On a une information supplémentaire par rapport à la commande `git status` précédemment : `Your branch is up to date with 'origin/main'`. Cela signifie que le dépôt local est à jour avec le dépôt distant.

Exemple de modifications et de push

On va maintenant faire quelques modifications sur le dépôt local et les pousser sur le dépôt distant. On va par exemple ajouter un fichier `exo.ts` dans le dossier `src` et modifier le fichier `README.txt` et on va faire un commit :

```
% touch src/exo.ts
% echo "Ceci est un fichier d'exercice" >> README.txt
% git add src/exo.ts README.txt
```

```
% git commit -m "feat: add exo.ts + modify README.txt"
[main 04542fa] feat: add exo.ts + modify README.txt
 2 files changed, 1 insertion(+)
 create mode 100644 src/exo.ts
```

On peut voir que le dépôt local a été mis à jour avec la commande `git status` :

```
% git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

On a une information supplémentaire par rapport à la commande `git status` précédemment : `Your branch is ahead of 'origin/main' by 1 commit.`. Cela signifie que le dépôt local est à jour avec le dépôt distant mais qu'il y a une différence entre les deux. On a donc fait un commit sur le dépôt local qui n'est pas encore sur le dépôt distant. Si vous allez sur le dépôt distant, vous verrez que le fichier `exo.ts` n'est pas présent et que le fichier `README.txt` n'a pas été modifié.

On peut donc pousser les modifications sur le dépôt distant avec la commande `git push` :

```
% git push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 10 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 451 bytes | 451.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
To https://gitlab.univ-lorraine.fr/jozefowi1/coursgit.git
   96ce157..04542fa  main -> main
```

On peut voir que le dépôt distant a bien été mis à jour et que l'ensemble des commits que l'on a fait sur le dépôt local ont été poussés sur le dépôt distant (cliquer sur `x commits`).

Remarque : on n'est pas obligé de faire un `git push` à chaque fois qu'on fait un commit. On peut faire plusieurs commits et ensuite faire un `git push` pour pousser tous les commits sur le dépôt distant. C'est ce que l'on a fait par exemple sur le push initial.

Ressource

Ce cours est largement basé sur le livre *Pro Git* disponible gratuitement en ligne à l'adresse https://git-scm.com/book/en/v2.